

SPE Projektarbeit zum Thema Biofeedback

SPE Projektarbeit zum Thema Biofeedback	1
1. Projektbeschreibung.....	1
2. Anforderungen	1
3. Projektumfeld	2
4. Schnittstellen.....	3
5. Durchführung.....	3
5.1. Integration und Vorbereitung des 3D-Herzens.....	3
5.2. Entwicklung der Herzsclaganimation	3
5.3. Steuerung der Animationsgeschwindigkeit	4
5.4. Dynamische Farbanpassung	5
5.5 Herausforderungen und Lösungsansätze	6
5.6. Vorgehensweise und Entscheidungen	6
5.7. Qualitätssicherung	7
5.8 . Future Work.....	7
6. Fazit und Lessons Learned	8
7. Übergabe.....	8

1. Projektbeschreibung

Im Rahmen des Projekts wurde ein einfacher Prototyp zur Darstellung eines menschlichen Herzschlags in Unity entwickelt. Ziel war es, die grundlegende Funktionsweise einer Biofeedback-Anwendung prototypisch umzusetzen. Hierfür wird ein dreidimensionales Herz dargestellt, dessen Größe sich entsprechend einer vorgegebenen Herzfrequenz verändert. Durch die pulsierende Bewegung soll ein menschlicher Herzschlag möglichst einfach und verständlich visualisiert werden.

Auftraggeberin des Projekts war Kerstin Metko, Trainerin bei der SPE. Die Anwendung richtet sich an zukünftige Entwickler, die den bestehenden Prototypen als Grundlage für die weitere Entwicklung verwenden können. Dabei sollen die Entwickler nicht nur die bisherige Umsetzung nachvollziehen können, sondern auch in der Lage sein, das Programm eigenständig weiterzuentwickeln, anzupassen und um zusätzliche Funktionen zu erweitern.

Das übergeordnete Ziel des Projekts bestand darin, einen ersten funktionsfähigen Prototypen mit Biofeedback-Bezug zu entwickeln. Dabei wurde besonderer Wert auf eine nachvollziehbare und wartbare Umsetzung gelegt. Die einzelnen Komponenten und Skripte sollten so aufgebaut werden, dass zukünftige Entwickler die Funktionsweise des Programms leicht nachvollziehen und die Anwendung bei Bedarf erweitern können.

Die Anwendung stellt zunächst einen theoretischen Herzschlag dar. Die Herzfrequenz wird dabei über den Parameter BPM (Beats per Minute) angegeben und bestimmt die Geschwindigkeit der Animation. Zusätzlich wurde die Möglichkeit umgesetzt, die Farbe des Herzens abhängig von der Herzfrequenz zu verändern. Bei niedrigeren Werten wird das Herz grün dargestellt, während es bei steigender Herzfrequenz zunächst gelb und anschließend orange bis rot wird.

2. Anforderungen

Die zentrale funktionale Anforderung des Projekts bestand darin, ein dreidimensionales Herz innerhalb einer Unity-Anwendung darzustellen und dieses mit einer pulsierenden Animation zu versehen. Das Herz soll sich dabei kurz vergrößern und anschließend wieder auf seine ursprüngliche Größe zurückkehren. Dadurch wird die Bewegung eines menschlichen Herzschlags vereinfacht dargestellt.

Eine weitere funktionale Anforderung bestand darin, die Geschwindigkeit der Animation an eine vorgegebene Herzfrequenz anzupassen. Als Parameter wird hierfür BPM (Beats per Minute) verwendet. Die Verwendung von BPM ermöglicht eine verständliche Zuordnung zwischen dem eingegebenen Wert und der Anzahl der dargestellten Herzschläge pro Minute. Bei einem Wert von beispielsweise 60 BPM soll das Herz theoretisch 60 Schläge pro Minute ausführen. Eine Erhöhung des BPM-Wertes führt entsprechend zu einer schnelleren Animation.

Als optionale Funktion sollte eine farbliche Rückmeldung des Herzens umgesetzt werden. Dabei sollte ein niedriger beziehungsweise normaler Herzfrequenzbereich durch die Farbe Grün dargestellt werden. Mit zunehmender Herzfrequenz sollte sich die Farbe über Gelb und Orange bis hin zu Rot verändern. Die rote Darstellung dient dabei als visuelle Warnung für eine besonders hohe Herzfrequenz.

Neben den funktionalen Anforderungen wurden auch nicht-funktionale Anforderungen berücksichtigt. Ein besonderer Schwerpunkt lag auf der Wartbarkeit und Nachvollziehbarkeit des Programms. Die verwendeten Skripte sollten übersichtlich aufgebaut und die Verbindung zwischen den einzelnen Unity-Komponenten verständlich gestaltet werden. Dadurch soll es zukünftigen Entwicklern ermöglicht werden, den bestehenden Prototypen ohne umfangreiche Einarbeitung zu verstehen und weiterzuentwickeln.

Von den im Projektantrag festgelegten Anforderungen wurde im Verlauf der Umsetzung nicht abgewichen. Sämtliche vorgesehenen Anforderungen konnten umgesetzt werden.

3. Projektumfeld

Die Entwicklung des Prototyps erfolgte vollständig innerhalb der Entwicklungsumgebung Unity. Als verwendete Version kam Unity 6.6 zum Einsatz. Die Programmierung der benötigten Funktionen wurde mit der Programmiersprache C# durchgeführt.

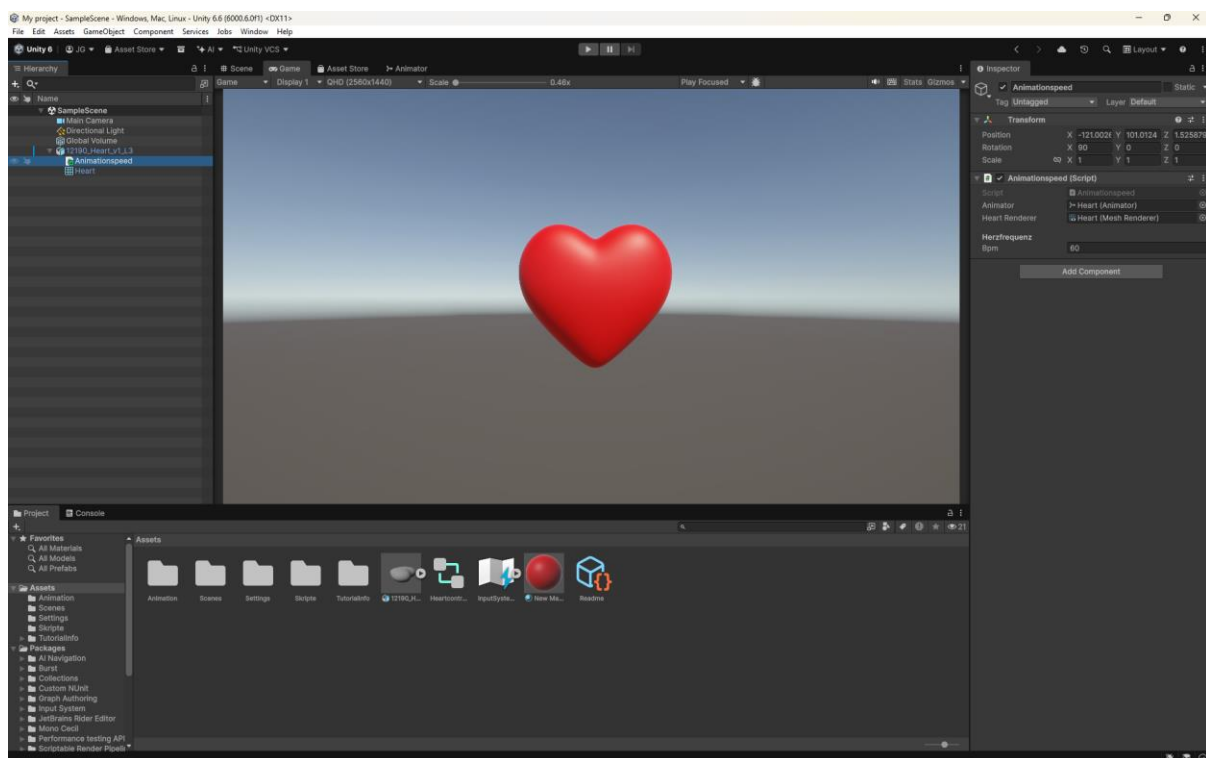


Abb. 1 Das Projekt in Unity

Für die Entwicklung wurde ein HP EliteBook verwendet. Die eigentliche Darstellung und Animation des Herzens erfolgt durch Unity. Die Steuerungslogik wurde über ein selbst erstelltes C#-Skript umgesetzt, welches mit den entsprechenden Unity-Komponenten des Herzobjekts verbunden wurde.

Für das dreidimensionale Herz wurde ein externes 3D-Modell verwendet. Das verwendete Asset stammt von Free3D und wurde anschließend innerhalb von Unity in das Projekt integriert. Das Modell ist unter folgender Quelle verfügbar:

<https://free3d.com/de/3d-model/heart-v1--539992.html>

Neben dem 3D-Herz wurden keine weiteren externen Assets benötigt. Die Animation, Steuerungslogik und Farbanpassung wurden eigenständig innerhalb von Unity umgesetzt.

4. Schnittstellen

Die aktuelle Version des Prototyps besitzt keine externe Datenbank- oder Dateischnittstelle. Die Eingabe der Herzfrequenz erfolgt direkt innerhalb des Unity Inspectors. Der dafür vorgesehene BPM-Parameter kann dort verändert werden und beeinflusst unmittelbar die Geschwindigkeit der Herzschlaganimation.

Die Verbindung zwischen dem Benutzerparameter und dem 3D-Herz erfolgt über ein C#-Skript. Dieses erhält den BPM-Wert und überträgt die daraus berechnete Geschwindigkeit an den Animator des Herzobjekts. Zusätzlich greift das Skript auf den Renderer des Herzens zu, um dessen Farbe abhängig von der eingestellten Herzfrequenz anzupassen.

Die aktuelle Bedienung ist bewusst auf den Unity Inspector beschränkt. Dies entspricht dem Charakter des Projekts als erster technischer Prototyp. Eine eigenständige Benutzeroberfläche wurde im Rahmen des Projekts nicht umgesetzt.

Für zukünftige Versionen wäre eine externe Datenschnittstelle denkbar. Beispielsweise könnten verschiedene Herzfrequenzwerte über eine CSV-Datei eingelesen werden. Eine weiterführende Möglichkeit wäre die direkte Anbindung eines Herzfrequenzmessers. Dadurch könnte die Anwendung nicht mehr nur vorgegebene Werte darstellen, sondern den tatsächlichen Herzschlag einer Person visualisieren.

5. Durchführung

5.1. Integration und Vorbereitung des 3D-Herzens

Zu Beginn der Entwicklung wurde zunächst ein geeignetes dreidimensionales Herzmodell in das Unity-Projekt integriert. Das Herzmodell wurde anschließend in der Szene positioniert und für die weitere Verarbeitung vorbereitet.

Um das Modell optisch hervorzuheben und eine eindeutige Darstellung zu ermöglichen, wurde dem Herz ein entsprechendes Material beziehungsweise eine rote Färbung zugewiesen. Die Darstellung des Herzens bildete anschließend die Grundlage für die weitere Entwicklung der Animation und der dynamischen Farbanpassung.



Abb. 2 Das 3D Herz mit Material

Da der Prototyp zunächst auf die grundlegende Funktionalität konzentriert wurde, wurde auf zusätzliche grafische Elemente und auf eine komplexe Umgebung verzichtet.

5.2. Entwicklung der Herzschlaganimation

Im nächsten Schritt wurde die eigentliche Herzschlaganimation umgesetzt. Ziel war es, eine möglichst einfache und nachvollziehbare Bewegung zu erzeugen, bei der das Herz kurzzeitig größer wird und anschließend wieder auf seine Ausgangsgröße zurückkehrt.

Hierfür wurde der Unity Animator verwendet. Die Skalierung des Herzobjekts wird innerhalb der Animation verändert. Ausgehend von einer Skalierung von 1 wird das Herz auf eine Skalierung von 1,5 vergrößert und anschließend wieder auf den Ausgangswert 1 zurückgesetzt.



Abb. 3 Animation des Herzens

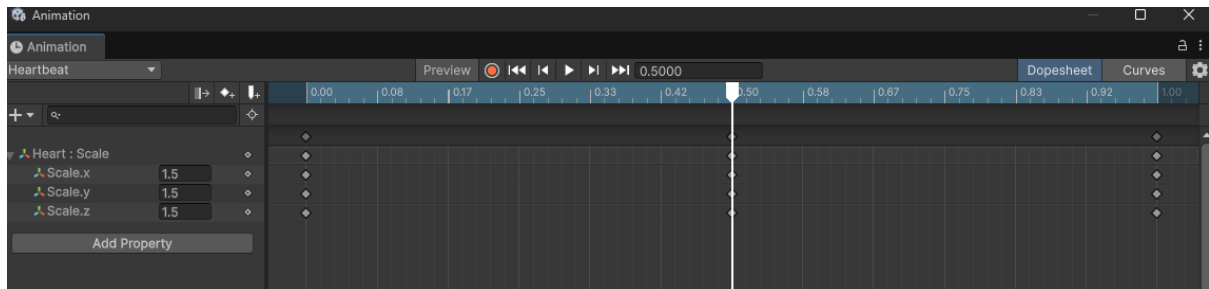


Abb. 4 Animation im Unity Animator

Die Animation besitzt eine Länge von einer Sekunde. Dadurch entspricht eine vollständige Animationswiederholung zunächst einer Herzfrequenz von 60 BPM. Diese Ausgangseinstellung wurde anschließend durch ein C#-Skript dynamisch steuerbar gemacht.

5.3. Steuerung der Animationsgeschwindigkeit

Nachdem die grundlegende Animation erstellt wurde, wurde die dynamische Steuerung der Animationsgeschwindigkeit implementiert. Hierfür wurde ein C#-Skript erstellt, welches mit dem Herzobjekt verbunden und über den Unity Inspector konfiguriert werden kann.

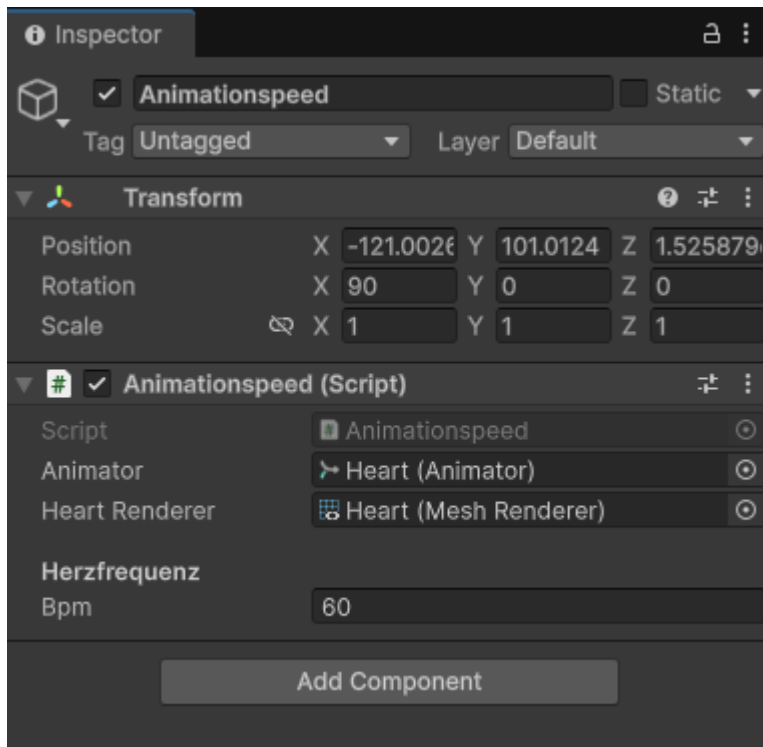


Abb. 5 Bild der Geschwindigkeitskontrolle

Ein zentraler Parameter des Skripts ist die Herzfrequenz bpm. Standardmäßig wird dieser auf 60 BPM gesetzt.

```
public Animator animator;
public Renderer heartRenderer;
```

```
[Header("Herzfrequenz")]
public float bpm = 60f;
```

```
void Update()
{
    // Geschwindigkeit der Herzschlag-Animation
    animator.speed = bpm / 60f;
}
```

Die Geschwindigkeit des Animators wird anhand des eingegebenen BPM-Wertes berechnet. Bei 60 BPM ergibt sich eine Animationsgeschwindigkeit von 1. Bei 120 BPM wird die Geschwindigkeit auf 2 erhöht, wodurch die Animation doppelt so schnell abläuft. Umgekehrt führt beispielsweise ein Wert von 30 BPM zu einer Animationsgeschwindigkeit von 0,5.

Durch diese Umsetzung kann die Geschwindigkeit der Herzschlaganimation flexibel über den BPM-Wert gesteuert werden, ohne dass für jede Herzfrequenz eine eigene Animation erstellt werden muss.

Das Skript wurde anschließend im Unity Inspector mit dem Animator Controller des Herzobjekts verknüpft. Dadurch kann der BPM-Wert verändert werden, während die eigentliche Animation unverändert bleibt.

5.4. Dynamische Farbanpassung

Als optionale Erweiterung wurde eine dynamische Anpassung der Farbe des Herzens implementiert. Die Farbe wird abhängig vom aktuell eingestellten BPM-Wert verändert und dient als zusätzliche visuelle Rückmeldung.

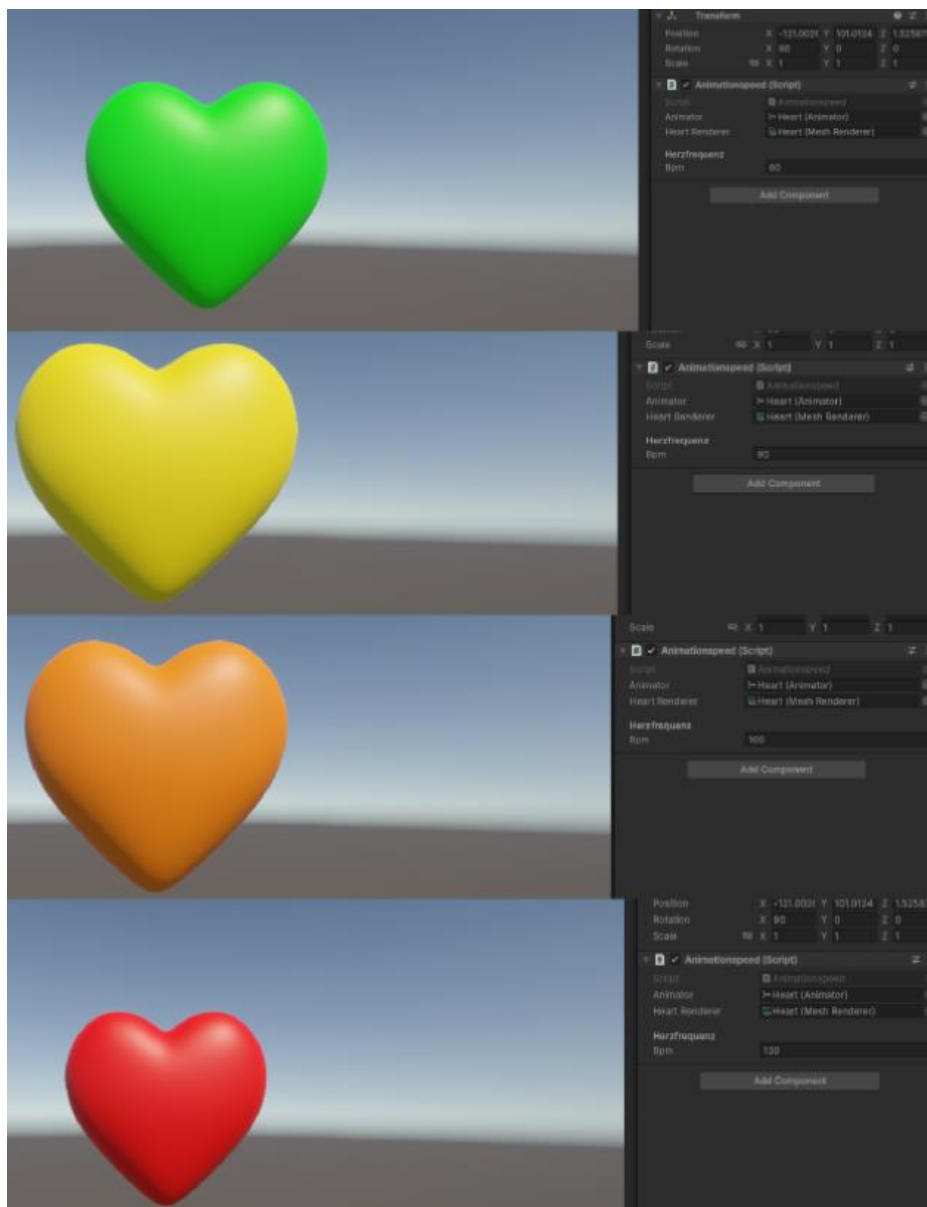


Abb. 6 Farbänderung bei verschiedenen BPM Werten

Hierfür wurde die bereits vorhandene Steuerungslogik des C#-Skripts erweitert. Über den Renderer des Herzobjekts kann dessen Materialfarbe verändert werden.

```
if (bpm < 80)
{
    heartRenderer.material.color = Color.green;
}
else if (bpm < 100)
{
    heartRenderer.material.color = Color.yellow;
}
else if (bpm < 130)
{
    heartRenderer.material.color = new Color(1f, 0.5f, 0f);
}
else
{
    heartRenderer.material.color = Color.red;
}
```

Die Umsetzung verwendet vier verschiedene Bereiche. Unterhalb von 80 BPM wird das Herz grün dargestellt. Ab 80 BPM wechselt die Darstellung zu Gelb. Ab 100 BPM wird Orange verwendet und ab 130 BPM wird das Herz rot dargestellt.

Dadurch erhält der Benutzer neben der Geschwindigkeit der Animation eine zusätzliche visuelle Rückmeldung über die eingestellte Herzfrequenz.

5.5 Herausforderungen und Lösungsansätze

Eine Herausforderung während der Umsetzung bestand in der korrekten Verknüpfung des C#-Skripts mit den Unity-Komponenten des Herzobjekts. Für die Steuerung der Animation musste das Skript Zugriff auf den Animator des Herzens erhalten. Zusätzlich wurde für die optionale Farbanpassung ein Renderer benötigt.

Eine weitere Herausforderung bestand in der korrekten Einrichtung des Animator Controllers. Die Herzsclaganimation musste so aufgebaut werden, dass sie kontinuierlich wiederholt werden kann und gleichzeitig über die Animationsgeschwindigkeit des Animators gesteuert wird.

Die Lösung bestand darin, die Animation unabhängig von der jeweiligen Herzfrequenz zu erstellen und die Geschwindigkeit anschließend ausschließlich über den BPM-

Parameter des C#-Skripts zu verändern. Dadurch musste die Animation selbst nicht für unterschiedliche Herzfrequenzen angepasst werden.

Durch diese Trennung zwischen Animation und Steuerungslogik wurde die Struktur des Prototyps übersichtlicher. Änderungen an der Herzfrequenz können dadurch vorgenommen werden, ohne die eigentliche Animation bearbeiten zu müssen.

5.6. Vorgehensweise und Entscheidungen

Das Projekt wurde schrittweise aufgebaut. Zunächst wurde das 3D-Herz in Unity integriert und für die Darstellung vorbereitet. Anschließend wurde die grundlegende Herzsclagananimation erstellt. Erst nachdem diese funktionsfähig war, wurde die dynamische Steuerung der Animationsgeschwindigkeit über ein C#-Skript ergänzt. Abschließend wurde die optionale Farbanpassung umgesetzt.

Die schrittweise Vorgehensweise wurde gewählt, um die einzelnen Funktionen unabhängig voneinander überprüfen zu können. Dadurch konnte zunächst sichergestellt werden, dass die Animation grundsätzlich funktioniert, bevor die zusätzliche Logik zur Steuerung und Farbdarstellung integriert wurde.

Auf eine umfangreiche Benutzeroberfläche wurde bewusst verzichtet. Da im Rahmen des Projekts zunächst lediglich ein funktionaler Prototyp entwickelt werden sollte und die verfügbare Entwicklungszeit begrenzt war, wurde die Steuerung über den Unity Inspector als ausreichend betrachtet. Der Schwerpunkt lag dadurch auf der technischen Umsetzung der Kernfunktion und nicht auf einer vollständigen Produktentwicklung.

Auch aus Sicht der Wartbarkeit wurde die Lösung bewusst einfach gehalten. Die Steuerungslogik befindet sich in einem zentralen C#-Skript und kann dadurch vergleichsweise leicht nachvollzogen und erweitert werden.

5.7. Qualitätssicherung

Zur Überprüfung der Funktionalität wurde die Herzsclagananimation mit unterschiedlichen BPM-Werten getestet. Dabei wurde ein Bereich von 0 bis 2000 BPM verwendet, um sowohl niedrige als auch sehr hohe Werte zu überprüfen.

Während der Tests konnten keine Fehler oder Fehlermeldungen bei der Ausführung der Animation festgestellt werden. Die Anpassung der Animationsgeschwindigkeit erfolgte entsprechend des eingegebenen BPM-Wertes.

Die Tests dienten insbesondere dazu zu überprüfen, ob die Verbindung zwischen dem BPM-Parameter und der Geschwindigkeit des Animator Controllers korrekt funktioniert.

Zusätzlich wurde die dynamische Farbanpassung anhand verschiedener BPM-Werte überprüft.

Da es sich um einen ersten technischen Prototypen handelt, lag der Schwerpunkt der Qualitätssicherung auf der grundlegenden technischen Funktionsfähigkeit und Nachvollziehbarkeit der Umsetzung.

5.8 . Future Work

Der entwickelte Prototyp stellt lediglich eine Grundlage für mögliche zukünftige Erweiterungen dar. Eine wesentliche Weiterentwicklung wäre die Anbindung eines echten Herzfrequenzmessers. Hierfür müsste die Anwendung um eine entsprechende Datenschnittstelle erweitert und das C#-Skript so angepasst werden, dass die gemessenen Werte automatisch als BPM-Werte übernommen werden.

Alternativ könnten Herzfrequenzwerte zunächst über eine CSV-Datei eingelesen werden. Dadurch wäre es möglich, mehrere Messwerte nacheinander abzuspielen und die Reaktion des Herzens auf unterschiedliche Herzfrequenzen zu untersuchen.

Bei einer zukünftigen Weiterentwicklung könnte die Anwendung beispielsweise dazu eingesetzt werden, Menschen ihren eigenen Herzschlag visuell zu verdeutlichen. Eine weitere mögliche Anwendung wäre die Unterstützung von Entspannungsübungen, bei denen Veränderungen der Herzfrequenz direkt visualisiert werden.

Darüber hinaus könnten eine eigenständige Benutzeroberfläche, weitere Visualisierungsmöglichkeiten sowie eine realistischere Herzbewegung ergänzt werden.

6. Fazit und Lessons Learned

Im Rahmen des Projekts wurde ein funktionaler Prototyp zur Visualisierung eines menschlichen Herzschlags in Unity entwickelt. Die im Projektantrag definierten Anforderungen konnten vollständig umgesetzt werden. Dazu gehören die Integration eines 3D-Herzens, die Erstellung einer pulsierenden Animation sowie die dynamische Anpassung der Animationsgeschwindigkeit anhand eines BPM-Wertes. Zusätzlich konnte die ursprünglich optionale farbliche Rückmeldung umgesetzt werden.

Während der Entwicklung wurden grundlegende Kenntnisse im Umgang mit Unity und dessen Animator-System aufgebaut und vertieft. Zusätzlich wurde die Verbindung zwischen Unity-Komponenten und selbst entwickelten C#-Skripten erarbeitet.

Ein besonderer Schwerpunkt lag auf der Wartbarkeit und Nachvollziehbarkeit. Durch die Trennung zwischen der eigentlichen Animation und der Steuerungslogik kann die

Anwendung vergleichsweise einfach angepasst und erweitert werden. Der BPM-Wert kann verändert werden, ohne die Animation selbst neu erstellen zu müssen.

Der aktuelle Stand stellt bewusst nur einen ersten Prototypen dar. Insbesondere die Anbindung realer Sensordaten und die Entwicklung einer eigenständigen Benutzeroberfläche bieten Möglichkeiten für zukünftige Erweiterungen.

7. Übergabe

Nach Abschluss der Entwicklung wird das Unity-Projekt über Git an die vorgesehenen Tester und den Prüfer übergeben. Dadurch steht der vollständige Projektstand einschließlich der verwendeten Skripte und Unity-Projektdateien zur Verfügung.

Die Verwendung von Git ermöglicht zudem eine nachvollziehbare Versionierung des Projekts. Änderungen am Projekt können dadurch dokumentiert und bei Bedarf nachvollzogen werden. Dies unterstützt gleichzeitig das übergeordnete Ziel einer wartbaren und nachvollziehbaren Projektstruktur.

Für die weitere Verwendung des Projekts sollte sichergestellt werden, dass die verwendete Unity-Version sowie die benötigten Projektdateien und Assets vorhanden sind. Dadurch kann der übergebene Projektstand bei zukünftigen Anpassungen erneut geöffnet und weiterentwickelt werden.